Université Paris Sorbonne Nord
Institut Galilée

Master 1.
Distributed Java Programming
TP5: Futures, CompletionService and Parallel Streams

Server

Consider the Server created in TP4 (offering some services to create new teachers and query the database).

1. Modify the class `Teacher` and add a new field `ID` with the needed *getter*.
2. Modify the server to offer a new method returning the list of all the IDs in the database.
3. Add a new method in the server that, given the ID of an employee, returns her/his salary. For the sake of the exercise, call `Thread.sleep` before retuning the salary[1].
4. Add a new method in the server that, given the ID of an employee, returns her/his name. For the sake of the exercise, call `Thread.sleep` before retuning the name.
5. Each connection established with the server must be handled in a separate thread (controlled by an `ExecutorService`).
6. During the same session/connection, the server may process several requests for the same client. Hence, the server expects an integer to signal the operation to be performed, e.g., 1. list of IDs ; 2. salary from a given ID ; 3. name from a given ID. If the server receives the integer `0`, the current session with the client is closed.
7. In order to perform some tests, add some teachers to the database in the `main` function that initializes the server.

Clients

Now we shall define a client that prints the employee payroll by using the services from the server. We shall implement different versions with different degrees of concurrency.

[1] You should implement the *database* as a `HashMap` instead of a `List` in order make easier the query of a Teacher given her/his ID.

1. **Sequential version** (from the client point of view). Implement a Client that first retrieves the list of IDs of all the employees. Then, for each employee, it queries the salary and the name of the employee and prints the payroll. Your client must open (and close) only one connection to the server.

```
ID |   Name     |  Salary
--------------------------
1  | Employee 1 |  $ 3.200
--------------------------
2  | Employee 2 |  $ 2.300
...
```

2. Each interaction with the server, needed to consult the name and salary of the employee, may take some time (specially due to the invocations to `Thread.sleep` in the server). Write a second client that:
   - Establish a connection to the server and obtains the list of all the IDs.
   - Using an `ExecutorService`, the client processes each ID in parallel (consulting the name and salary and then printing the record).

   Note that in this Client, several connection to the server are opened: one for retrieving the list of IDs and one for each ID (to consult the name and salary).

3. Modify the previous client as follows:
   - Define an instance of `Callable<Double>` that opens a connection with the server, consults the salary of a given ID and then closes the connection.
   - Define an instance of `Callable<String>` for consulting the salary of an ID.
   - Now use the previous *Callable* to asynchronously obtain, as a `Future` the two needed values. When they are available (method `get` of `Future`), the information of the employee is printed.

   Note that this version of the client opens even more connection to the server. Be careful of closing all the sessions with the server by sending the option `0`.

4. Now define a simpler client that, given an ID, performs the two queries sequentially and prints the information of the employee. Then, use a *Parallel Stream* to process the whole list.

5. Compare the four implementations. Use the command `time` in linux to measure the time needed for each implementation.